

Java Programming Practice Set

Beginner → Intermediate → Advanced • Curated from the concepts currently covered in your Notes repository

Repository area	Observed coverage
Topics	Java introduction/JVM/compilation; primitive types; literals; variables; operators; Scanner; classes/library usage; methods; strings.
Practice	Arithmetic, quadratic roots, bitwise operations, masking/merging, regex validation, conditionals, switch, menu programs, loops, digit problems, Armstrong, reverse, palindrome, etc.
This PDF	72 new practice problems arranged as a progression. Problems intentionally avoid simply repeating the repository's existing examples.

Repository scope reviewed

The current public repository contains a Java Fundamental section with six topic notes plus practice/self-practice material. The covered material includes Java introduction/JVM and compilation, primitive types and literals, variables and naming, arithmetic/relational/logical/conditional/bitwise operators, Scanner input, System.out formatting, Math utilities, classes/library usage, methods, and substantial String operations including indexing, substring, replace, split/join, validation with regex, and related processing.

Important calibration

The repository is still at the Java-fundamentals stage. Therefore, the 'Advanced' section below means advanced relative to the concepts currently covered in the repository—not advanced DSA, competitive programming, multithreading, collections, or Spring.

How to use this sheet

Solve in order. Do not immediately look for a solution. For each problem, first write the input/output contract, then a small dry run, then code it, then test edge cases. When a problem feels easy, solve it again without looking at your previous implementation.

Recommended progression

Beginner: build fluency with input, operators, conditionals, loops, digits and basic String methods. **Intermediate:** combine multiple concepts and start writing reusable methods. **Advanced:** build small console systems and solve multi-step parsing, regex, bitwise, and class-based problems.

Beginner

Target: you should be able to solve most problems with Scanner, variables, operators, if/else, switch, loops, Math, char and basic String methods.

B01 — Rectangle Calculator

Concepts: Input + arithmetic

Read length and breadth. Print area and perimeter. Reject non-positive dimensions.

B02 — Temperature Converter

Concepts: Input + arithmetic

Read Celsius and print Fahrenheit and Kelvin. Use floating-point arithmetic.

B03 — Three-Number Statistics

Concepts: Variables + arithmetic

Read three numbers and print their sum, average, largest, and smallest.

B04 — Profit or Loss

Concepts: Operators

Read cost price and selling price. Report profit/loss amount and percentage.

B05 — Time Converter

Concepts: Operators

Read total seconds and convert them into hours, minutes, and remaining seconds.

B06 — Character Inspector

Concepts: char + operators

Read one character and print its numeric Unicode value, previous character, next character, and whether it is a letter, digit, or other symbol.

B07 — Even/Odd + Sign

Concepts: Conditionals

Read an integer and report whether it is positive, negative, or zero and whether it is even or odd.

B08 — Largest of Three

Concepts: Conditionals

Read three integers and find the largest without using Math.max(). Handle ties clearly.

B09 — Leap-Year Validator

Concepts: Conditionals

Read a year and determine whether it is a leap year using the complete Gregorian rule.

B10 — Day Number

Concepts: switch

Read 1–7 and print the corresponding day. Print an error for every other input.

B11 — Simple Calculator

Concepts: switch + arithmetic

Read two numbers and an operation (+, -, *, /, %). Perform the selected operation and handle division by zero.

B12 — Print Multiples

Concepts: Loops

Read n and print the first 20 multiples of n in a clean table.

B13 — Sum 1..N

Concepts: Loops

Read n and calculate $1 + 2 + \dots + n$. Test $n = 0$ and negative input.

B14 — Factorial

Concepts: Loops

Read n and calculate $n!$ using a loop. Use long and state the practical input limit you choose.

B15 — Digit Sum

Concepts: Loops + digits

Read an integer and calculate the sum of its decimal digits without converting it to String.

B16 — Digit Count

Concepts: Loops + digits

Count the number of decimal digits in an integer. Decide how your program treats 0 and negative values.

B17 — Reverse Number

Concepts: Loops + digits

Reverse an integer using % and /. Preserve the sign.

B18 — Palindrome Number

Concepts: Loops + conditionals

Determine whether an integer reads the same forwards and backwards.

B19 — Prime Checker

Concepts: Loops + conditionals

Read n and determine whether it is prime. Do not check every number up to n if you can avoid unnecessary work.

B20 — Primes in a Range

Concepts: Loops

Read two bounds and print every prime between them.

B21 — Quadratic Equation

Concepts: Math

Read a , b , c . Compute the discriminant and report two real roots, one repeated real root, or no real roots.

B22 — String Length Without length()

Concepts: String basics

Read a String and determine its length by iterating over characters.

B23 — First and Last Character

Concepts: String methods

Read a non-empty String and print its first character, last character, and their Unicode values.

B24 — Basic Email Split

Concepts: String methods

Given an email-like string, extract the username and domain using `indexOf()` and `substring()`. Also report whether '@' exists.

Intermediate

Target: combine concepts, write methods, process complete lines, use regex and bitwise operations, and reason about edge cases.

I01 — Armstrong Number

Concepts: Loops + arithmetic

Check whether a number is an Armstrong number. First support 3-digit numbers; then modify your solution to work for any digit count.

I02 — Perfect Number

Concepts: Loops + arithmetic

Check whether n equals the sum of its proper positive divisors.

I03 — GCD and LCM

Concepts: Loops + arithmetic

Read two positive integers and calculate their GCD and LCM. Handle cases involving 1.

I04 — Second Largest

Concepts: Loops + arithmetic

Read three distinct integers and determine the second-largest without sorting.

I05 — Fibonacci Series

Concepts: Loops

Print the first n Fibonacci numbers iteratively. Avoid recursive solutions.

I06 — Number Triangle

Concepts: Nested loops

Print a right-aligned triangle of numbers for n rows. Then create a second version with reversed rows.

I07 — Multiplication Grid

Concepts: Nested loops

Print a multiplication table from 1 to n as a formatted square grid.

I08 — Prime Factorization

Concepts: Loops + conditionals

Print the prime factors of n with repetition, e.g. $60 \rightarrow 2\ 2\ 3\ 5$.

I09 — Frequency of Digits

Concepts: Loops + digits

For a non-negative integer, count how many times each digit 0–9 occurs.

I10 — Second Digit from Right

Concepts: Loops + digits

Extract the second digit from the right using arithmetic only. Extend it to extract the k th digit.

I11 — Reusable Number Utilities

Concepts: Methods

Create separate methods for `isPrime()`, `reverse()`, `digitSum()`, and `isPalindrome()`. Build a menu that calls them.

I12 — Geometry Utility

Concepts: Methods + Math

Create methods for circle area/circumference, rectangle area/perimeter, and cuboid surface area/volume. Validate inputs.

I13 — Calculator Overload

Concepts: Methods + overloading

Create overloaded methods for int, long, and double addition/multiplication. Demonstrate how Java selects the method.

I14 — Count Character Types

Concepts: String traversal

Read a full line and count uppercase letters, lowercase letters, digits, spaces, and special characters.

I15 — Manual String Reverse

Concepts: String traversal

Reverse a String without using reverse() or StringBuilder. Preserve spaces and punctuation.

I16 — Palindrome String

Concepts: String methods

Check whether a String is a palindrome while ignoring case. Then create a version that also ignores spaces.

I17 — Vowel/Consonant Analyzer

Concepts: String methods

Count vowels, consonants, digits, and other characters in a line of text.

I18 — Remove Duplicate Spaces

Concepts: String methods

Replace every run of whitespace with one space and trim leading/trailing whitespace.

I19 — Binary Validator

Concepts: Regex

Validate whether an input contains only binary digits. If valid, also convert it to decimal without using Integer.parseInt(binary, 2).

I20 — Integer Validator

Concepts: Regex

Validate signed decimal integers such as -25, 0, and +17 while rejecting malformed inputs such as 12-3 or ++5.

I21 — Date Shape Validator

Concepts: Regex

Validate the shape dd/mm/yyyy. Then improve the program so that impossible day/month combinations are rejected.

I22 — Website Analyzer

Concepts: String parsing

Given a URL-like String, extract protocol, host, domain extension, and optionally the path. Handle missing components.

I23 — Username/Domain Extractor

Concepts: String parsing

Given an email address, extract username, domain name, and extension. Reject inputs with multiple or missing '@' separators.

I24 — Word Counter

Concepts: String methods

Read a full sentence and count words without using split(). Treat multiple spaces as one separator.

I25 — Longest Word

Concepts: String methods

Read a sentence and determine its longest word without using split().

I26 — Character Frequency

Concepts: String methods

Find the most frequent character in a line, ignoring spaces and case. If tied, report all tied characters.

127 — Caesar Shift

Concepts: String transformation

Implement a Caesar cipher for alphabetic characters with a user-supplied shift. Preserve case and leave non-letters unchanged.

128 — Set/Clear/Toggle Bit

Concepts: Bitwise

Read an integer and a bit position. Create methods to set, clear, toggle, and test that bit using bitwise operators.

129 — Power of Two

Concepts: Bitwise

Determine whether a positive integer is a power of two using bitwise logic. Explain why the test works.

130 — Pack Two Small Numbers

Concepts: Bitwise packing

Pack two 4-bit values into one integer and later extract both values using shifts and masks. Reject values outside 0–15.

Advanced

Target: design the solution yourself, split the program into methods/classes, validate input, and combine several repository concepts in one program.

A01 — Number Utility Console

Concepts: Methods + loops

Build a menu-driven utility containing prime test, factorial, GCD, LCM, digit sum, reverse, palindrome, Armstrong, and perfect-number operations. Each operation must be a separate method.

A02 — Robust Calculator

Concepts: Methods + validation

Build a calculator that repeatedly accepts input until the user chooses Exit. Validate numeric input, operation choice, and division by zero.

A03 — Password Strength Analyzer

Concepts: String + regex

Given a password, report whether it meets configurable rules: minimum length, uppercase, lowercase, digit, special character, and no whitespace. Use String methods and regex where appropriate.

A04 — Email Validator

Concepts: String + regex

Create a practical email validator using a regex you design yourself. Explain what your regex accepts and rejects; do not claim it implements the full email RFC.

A05 — URL Parser Lite

Concepts: String parsing

Parse a URL-like input into protocol, host, optional port, path, query, and extension using indexOf(), lastIndexOf(), substring(), and conditionals.

A06 — Run-Length Encoding

Concepts: String algorithms

Compress a string using run-length encoding, e.g. aaabbc -> a3b2c1. Also implement decompression for valid encoded input.

A07 — Manual Split

Concepts: String algorithms

Implement your own split-by-space operation without String.split(). Return/process each word and preserve the behavior for repeated spaces according to rules you define.

A08 — Anagram Checker

Concepts: String algorithms

Determine whether two strings are anagrams while ignoring case and spaces. Do not use advanced collection classes; use arrays or character counting.

A09 — First Non-Repeating Character

Concepts: String algorithms

Find the first character that occurs exactly once, ignoring spaces and case. Report its original character.

A10 — Prime Pair Search

Concepts: Loops + nested loops

For a user-supplied limit n, print all pairs of primes whose sum equals n. Then modify it to find such pairs for every even number in a range.

A11 — Permission Flags

Concepts: Bitwise

Represent READ, WRITE, EXECUTE, and ADMIN permissions as bits in one integer. Implement grant, revoke, toggle, and hasPermission methods.

A12 — Pack Three Values

Concepts: Bitwise packing

Pack three values with known bit widths into one integer. Extract each value correctly using shifts and masks.

A13 — Count Set Bits

Concepts: Bitwise

Count the number of 1-bits in an integer. First use repeated shifting, then implement a second version using $n \& (n - 1)$.

A14 — Bitwise Binary Inspector

Concepts: Bitwise

For an integer, print its binary representation and report whether selected bit positions are set. Include negative-input behavior in your design.

A15 — Mini Banking Console

Concepts: Methods + switch

Build a console banking simulation with deposit, withdrawal, balance, and transaction-menu methods. Validate amounts and prevent invalid withdrawals.

A16 — Student Result System

Concepts: Methods + strings

Read student name and marks for several subjects, calculate total/average, assign a grade, and print a formatted result using methods.

A17 — BankAccount Class

Concepts: Classes + methods

Create a BankAccount class with private balance, account holder, deposit(), withdraw(), and displayBalance(). Enforce valid operations through methods.

A18 — Contact Record

Concepts: Classes + methods + strings

Create a Contact class storing name, phone, and email. Add methods for validation and formatted display. Use String/regex validation from the repository topics.

Practice Protocol

- 1. Understand:** Write down exactly what the program receives and what it must print.
- 2. Dry run:** Use at least two normal cases and two edge cases before coding.
- 3. Implement:** Write the simplest correct version using only concepts you have already studied.
- 4. Refactor:** For intermediate/advanced problems, move repeated logic into methods and use meaningful names.
- 5. Break it:** Try zero, negative values, empty strings, very large values, invalid choices, repeated spaces, and boundary inputs where relevant.
- 6. Explain:** After solving, explain why your algorithm works and what its time/space cost is, even if the problem is small.

Milestone targets

After B01–B08	Basic input, arithmetic, comparison, and conditional fluency.
After B09–B16	Comfort with switch, loops, factorial, digit extraction and numeric reasoning.
After B17–B24	Solid basic problem-solving with numbers and Strings.
After I01–I13	Ability to decompose problems into reusable methods.
After I14–I30	Strong String, regex and bitwise fundamentals.
After A01–A18	Ability to design small Java console programs rather than only solve isolated questions.

Note: Do not treat completion count as the goal. If you cannot solve a problem without copying the approach, it is not mastered yet.